

Ini adalah penjelasan mendalam mengenai kode C# tersebut.

Gambaran Umum

Kode ini mengimplementasikan algoritma **Linear Contrast Stretching** (Peregangan Kontras Linear). Tujuannya adalah untuk meningkatkan kontras gambar dengan "merenggangkan" rentang nilai intensitas piksel (RGB) agar memenuhi seluruh spektrum dari 0 hingga 255.

Teknik yang digunakan adalah **Direct Memory Access (LockBits)**. Ini jauh lebih cepat daripada menggunakan GetPixel dan SetPixel bawaan, karena kita memanipulasi *byte array* di memori secara langsung, ditambah dengan penggunaan **Parallel Processing** untuk memanfaatkan multi-core CPU.

Penjelasan Per Baris

Berikut adalah rincian logika kode tersebut:

1. Setup Awal dan Pembuatan Bitmap Tujuan

C#

```
int w = source.Width;  
int h = source.Height;  
Bitmap dst = new Bitmap(w, h, PixelFormat.Format32bppArgb);
```

- Kita mengambil lebar (w) dan tinggi (h) dari gambar sumber.
- Kita membuat objek Bitmap baru bernama dst (destination) dengan ukuran yang sama.
- **Penting:** Format piksel diatur ke Format32bppArgb. Ini berarti setiap piksel memakan 4 byte (Blue, Green, Red, Alpha).

2. Mengunci Memori (LockBits)

C#

```
BitmapData srcData = source.LockBits(  
    new Rectangle(0, 0, w, h),  
    ImageLockMode.ReadOnly,
```

```
PixelFormat.Format32bppArgb);
```

```
BitmapData dstData = dst.LockBits(  
    new Rectangle(0, 0, w, h),  
    ImageLockMode.WriteOnly,  
    PixelFormat.Format32bppArgb);
```

- LockBits: Fungsi ini "mengunci" data gambar di memori sistem agar kita bisa mengakses alamat memorinya secara langsung.
- srcData (Sumber): Dikunci dalam mode ReadOnly karena kita hanya akan membacanya.
- dstData (Tujuan): Dikunci dalam mode WriteOnly karena kita akan menulis hasil editan ke sini.

3. Menyalin Data ke Array Byte

C#

```
int bytes = Math.Abs(srcData.Stride) * h;  
byte[] buffer = new byte[bytes];  
byte[] result = new byte[bytes];  
Marshal.Copy(srcData.Scan0, buffer, 0, bytes);
```

- **Stride**: Lebar satu baris gambar dalam satuan *byte*. bytes adalah total ukuran data gambar dalam memori.
- buffer: Array byte untuk menampung data gambar asli.
- result: Array byte untuk menampung hasil editan.
- Marshal.Copy: Ini menyalin data dari pointer memori tak terkelola (*unmanaged memory* Scan0) ke dalam array buffer yang terkelola (*managed*) agar bisa diproses oleh C#.

4. Mencari Nilai Min dan Max (Analisis Histogram)

C#

```
byte minR = 255, maxR = 0;  
// ... inisialisasi min/max untuk G dan B ...  
  
for (int i = 0; i < bytes; i += 4)  
{
```

```

byte b = buffer[i];
byte g = buffer[i + 1];
byte r = buffer[i + 2];
// ... logika if untuk mencari min/max ...
}

```

- Loop ini memindai seluruh piksel gambar.
- $i += 4$: Kita melompat 4 langkah setiap kali karena formatnya 32bpp (4 byte per piksel).
- Dalam memori Windows (Little Endian), urutannya biasanya **BGRA**:
 - `buffer[i]`: Blue
 - `buffer[i+1]`: Green
 - `buffer[i+2]`: Red
 - `buffer[i+3]`: Alpha (tidak digunakan dalam perhitungan min/max).
- Loop ini mencari piksel paling gelap (min) dan paling terang (max) untuk masing-masing warna.

5. Mencegah Pembagian dengan Nol

C#

```

if (maxB == minB) maxB++;
if (maxG == minG) maxG++;
if (maxR == minR) maxR++;

```

- Jika gambar berupa warna solid (misal: biru rata), maka `maxB` akan sama dengan `minB`.
- Rumus peregangan melibatkan pembagian ($\text{max} - \text{min}$). Jika hasilnya 0, program akan *crash* (`DivideByZeroException`). Kode ini mencegah hal itu dengan sedikit menaikkan nilai `max`.

6. Menerapkan Rumus Peregangan (Parallel Processing)

C#

```

Parallel.For(0, bytes / 4, i =>
{
    int k = i * 4;
    result[k] = (byte)((buffer[k] - minB) * 255 / (maxB - minB)); // Blue
    result[k + 1] = (byte)((buffer[k + 1] - minG) * 255 / (maxG - minG)); // Green

```

```

    result[k + 2] = (byte)((buffer[k + 2] - minR) * 255 / (maxR - minR)); // Red
    result[k + 3] = buffer[k + 3]; // Alpha (Salin saja)
});

```

- Parallel.For: Menggunakan multi-threading. Jika Anda memiliki CPU 8-core, pekerjaan ini akan dibagi ke core tersebut agar sangat cepat.
- Rumus Matematika:
Ini adalah implementasi rumus normalisasi:

$$P_{\text{baru}} = (P_{\text{lama}} - \text{Min}) \times \frac{255}{\text{Max} - \text{Min}}$$

- $\text{buffer}[k] - \text{min}$: Menggeser histogram supaya nilai terendah dimulai dari 0.
- $* 255 / (\text{max} - \text{min})$: Menskalakan (merenggangkan) nilai supaya nilai tertinggi menyentuh 255.

7. Menyimpan Kembali dan Membersihkan Memori

C#

```

Marshal.Copy(result, 0, dstData.Scan0, bytes);
source.UnlockBits(srcData);
dst.UnlockBits(dstData);
return dst;

```

- Marshal.Copy: Menyalin data hasil (result) kembali ke memori gambar tujuan (dstData.Scan0).
- UnlockBits: **Sangat penting.** Ini melepaskan kunci memori. Jika lupa, memori akan bocor atau gambar tidak bisa digunakan oleh proses lain.
- return dst: Mengembalikan gambar baru yang kontrasnya sudah diperbaiki.

Ringkasan Visual

Konsep utamanya adalah jika sebuah gambar terlihat "pucat" (misalnya nilai piksel hanya berkisar antara 50 hingga 150), kode ini akan menarik nilai 50 menjadi 0, dan nilai 150 menjadi 255, sehingga gambar terlihat lebih tajam dan kontras.

Apakah Anda ingin saya menjelaskan bagaimana cara memodifikasi kode ini untuk menangani *grayscale* saja atau menambahkan fitur *thresholding*?